

PROJET 4 DATA ANALYST

Réalisez une étude de santé publique avec R ou Python

OBJECTIF DE CE NOTEBOOK

Bienvenue dans l'outil plébiscité par les analystes de données Jupyter.

Il s'agit d'un outil permettant de mixer et d'alterner codes, textes et graphique.

Cet outil est formidable pour plusieurs raisons:

- il permet de tester des lignes de codes au fur et à mesure de votre rédaction, de constater immédiatement le résultat d'une instruction, de la corriger si nécessaire.
- De rédiger du texte pour expliquer l'approche suivie ou les résultats d'une analyse et de le mettre en forme grâce à du code html ou plus simple avec **Markdown**
- d'agrémenter de graphiques

Pour vous aider dans vos premiers pas à l'usage de Jupyter et de Python, nous avons rédigé ce notebook en vous indiquant les instructions à suivre.

Il vous suffit pour cela de saisir le code Python répondant à l'instruction donnée.

Vous verrez de temps à autre le code Python répondant à une instruction donnée mais cela est fait pour vous aider à comprendre la nature du travail qui vous est demandée.

Et garder à l'esprit, qu'il n'y a pas de solution unique pour résoudre un problème et qu'il y a autant de résolutions de problèmes que de développeurs ;)...

Etape 1 - Importation des librairies et chargement des fichiers

1.1 - Importation des librairies

```
In [1]: #Importation de la Librairie Pandas
import pandas as pd # Importation de la Librairie Pandas
import numpy as np # Importation de la Librairie Numpy
```

```
import matplotlib.pyplot as plt # Importation de la Librairie Matplotlib pour Les g
import seaborn as sns # Importation de la Librairie Seaborn pour Les graphiques
```

1.2 - Chargement des fichiers Excel

```
In [2]: #Importation du fichier population.csv
population = pd.read_csv('population.csv')

#Importation du fichier dispo_alimentaire.csv
dispo_alimentaire= pd.read_csv('dispo_alimentaire.csv')

#Importation du fichier aide_alimentaire.csv
aide_alimentaire = pd.read_csv('aide_alimentaire.csv')

#Importation du fichier sous_nutrition.csv
sous_nutrition = pd.read_csv('sous_nutrition.csv')
```

Etape 2 - Analyse exploratoire des fichiers

2.1 - Analyse exploratoire du fichier population

```
In [3]: #Afficher les dimensions du dataset
print("Le tableau comporte {} observation(s) ou article(s)".format(population.shape[0]))
print("Le tableau comporte {} colonne(s)".format(population.shape[1]))
```

Le tableau comporte 1416 observation(s) ou article(s)

Le tableau comporte 3 colonne(s)

```
In [4]: #Consulter le nombre de colonnes
print("nombre de colonnes:", population.info())
#La nature des données dans chacune des colonnes
print("Nature des données dans chacune des colonnes:", population.dtypes)
#Le nombre de valeurs présentes dans chacune des colonnes
print("Nombre de valeurs présentes dans chacune des colonnes:", population.count())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1416 entries, 0 to 1415
Data columns (total 3 columns):
#   Column  Non-Null Count  Dtype
---  -
0   Zone    1416 non-null     object
1   Année   1416 non-null     int64
2   Valeur  1416 non-null     float64
dtypes: float64(1), int64(1), object(1)
memory usage: 33.3+ KB
nombre de colonnes: None
Nature des données dans chacune des colonnes: Zone          object
Année          int64
Valeur        float64
dtype: object
Nombre de valeurs présentes dans chacune des colonnes: Zone          1416
Année          1416
Valeur          1416
dtype: int64
```

In [5]: *#Affichage Les 5 premières lignes de la table*
 population.head()

Out[5]:

	Zone	Année	Valeur
0	Afghanistan	2013	32269.589
1	Afghanistan	2014	33370.794
2	Afghanistan	2015	34413.603
3	Afghanistan	2016	35383.032
4	Afghanistan	2017	36296.113

In [6]: *#Nous allons harmoniser Les unités. Pour cela, nous avons décidé de multiplier la p*
#Multiplication de la colonne valeur par 1000

```
population['Valeur'] = population['Valeur'] * 1000
population.head()
```

Out[6]:

	Zone	Année	Valeur
0	Afghanistan	2013	32269589.0
1	Afghanistan	2014	33370794.0
2	Afghanistan	2015	34413603.0
3	Afghanistan	2016	35383032.0
4	Afghanistan	2017	36296113.0

In [7]: *#changement du nom de la colonne Valeur par Population*
 population.rename(columns={'Valeur': 'Population'}, inplace=True)

In [8]: *#Affichage les 5 premières lignes de la table pour voir les modifications*
 population.head()

Out[8]:

	Zone	Année	Population
0	Afghanistan	2013	32269589.0
1	Afghanistan	2014	33370794.0
2	Afghanistan	2015	34413603.0
3	Afghanistan	2016	35383032.0
4	Afghanistan	2017	36296113.0

2.2 - Analyse exploratoire du fichier disponibilité alimentaire

In [9]: *#Afficher les dimensions du dataset*
 print("le tableau comporta {} observations".format(dispo_alimentaire.shape[0]))
 print("le tableau comporta {} colonnes".format(dispo_alimentaire.shape[1]))

le tableau comporta 15605 observations
 le tableau comporta 18 colonnes

In [10]: *#Consulter le nombre de colonnes*
 print("nombre de colonnes:", dispo_alimentaire.info())
#La nature des données dans chacune des colonnes
 print("Nature des données dans chacune des colonnes:", dispo_alimentaire.dtypes)
#Le nombre de valeurs présentes dans chacune des colonnes
 print("Nombre de valeurs présentes dans chacune des colonnes:", dispo_alimentaire.c


```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 15605 entries, 0 to 15604
Data columns (total 18 columns):
  #   Column                                     Non-Null Count
Dtype  -----
-----
  0   Zone                                     15605 non-null
object
  1   Produit                                 15605 non-null
object
  2   Origine                                15605 non-null
object
  3   Aliments pour animaux                  2720 non-null
float64
  4   Autres Utilisations                    5496 non-null
float64
  5   Disponibilité alimentaire (Kcal/personne/jour) 14241 non-null
float64
  6   Disponibilité alimentaire en quantité (kg/personne/an) 14015 non-null
float64
  7   Disponibilité de matière grasse en quantité (g/personne/jour) 11794 non-null
float64
  8   Disponibilité de protéines en quantité (g/personne/jour) 11561 non-null
float64
  9   Disponibilité intérieure                15382 non-null
float64
 10   Exportations - Quantité                12226 non-null
float64
 11   Importations - Quantité               14852 non-null
float64
 12   Nourriture                            14015 non-null
float64
 13   Pertes                               4278 non-null
float64
 14   Production                            9180 non-null
float64
 15   Semences                             2091 non-null
float64
 16   Traitement                           2292 non-null
float64
 17   Variation de stock                    6776 non-null
float64
dtypes: float64(15), object(3)
memory usage: 2.1+ MB
nombre de colonnes: None
Nature des donnees dans chacune des colonnes: Zone
object
Produit                                     object
Origine                                    object
Aliments pour animaux                     float64
Autres Utilisations                       float64
Disponibilité alimentaire (Kcal/personne/jour) float64
Disponibilité alimentaire en quantité (kg/personne/an) float64
Disponibilité de matière grasse en quantité (g/personne/jour) float64
Disponibilité de protéines en quantité (g/personne/jour) float64

```

```

Disponibilité intérieure float64
Exportations - Quantité float64
Importations - Quantité float64
Nourriture float64
Pertes float64
Production float64
Semences float64
Traitement float64
Variation de stock float64
dtype: object
Nombre de valeurs présentes dans chacune des colonnes: Zone
15605
Produit 15605
Origine 15605
Aliments pour animaux 2720
Autres Utilisations 5496
Disponibilité alimentaire (Kcal/personne/jour) 14241
Disponibilité alimentaire en quantité (kg/personne/an) 14015
Disponibilité de matière grasse en quantité (g/personne/jour) 11794
Disponibilité de protéines en quantité (g/personne/jour) 11561
Disponibilité intérieure 15382
Exportations - Quantité 12226
Importations - Quantité 14852
Nourriture 14015
Pertes 4278
Production 9180
Semences 2091
Traitement 2292
Variation de stock 6776
dtype: int64

```

```

In [11]: #Affichage Les 5 premières lignes de la table
dispo_alimentaire.head()

```

```

Out[11]:

```

	Zone	Produit	Origine	Aliments pour animaux	Autres Utilisations	Disponibilité alimentaire (Kcal/personne/jour)	Dispo alimen c (kg/perso)
--	------	---------	---------	-----------------------------	------------------------	--	------------------------------------

0	Afghanistan	Abats Comestible	animale	NaN	NaN	5.0	
1	Afghanistan	Agrumes, Autres	vegetale	NaN	NaN	1.0	
2	Afghanistan	Aliments pour enfants	vegetale	NaN	NaN	1.0	
3	Afghanistan	Ananas	vegetale	NaN	NaN	0.0	
4	Afghanistan	Bananes	vegetale	NaN	NaN	4.0	

```

In [12]: #remplacement des NaN dans le dataset par des 0
dispo_alimentaire.fillna(0, inplace=True)

```

```
dispo_alimentaire.head()
```

Out[12]:

	Zone	Produit	Origine	Aliments pour animaux	Autres Utilisations	Disponibilité alimentaire (Kcal/personne/jour)	Dispo alimen c (kg/perso
0	Afghanistan	Abats Comestible	animale	0.0	0.0	5.0	
1	Afghanistan	Agrumes, Autres	vegetale	0.0	0.0	1.0	
2	Afghanistan	Aliments pour enfants	vegetale	0.0	0.0	1.0	
3	Afghanistan	Ananas	vegetale	0.0	0.0	0.0	
4	Afghanistan	Bananes	vegetale	0.0	0.0	4.0	

```
In [13]: colonnes_milliers_tonnes = [
    'Aliments pour animaux', 'Autres Utilisations', 'Disponibilité intérieure',
    'Exportations - Quantité', 'Importations - Quantité', 'Nourriture',
    'Pertes', 'Production', 'Semences', 'Traitement', 'Variation de stock'
]
dispo_alimentaire[colonnes_milliers_tonnes] = dispo_alimentaire[colonnes_milliers_t
```

```
In [14]: #Affichage Les 5 premières Lignes de La table
dispo_alimentaire.head()
```

Out[14]:

	Zone	Produit	Origine	Aliments pour animaux	Autres Utilisations	Disponibilité alimentaire (Kcal/personne/jour)	Dispo alimen c (kg/perso
0	Afghanistan	Abats Comestible	animale	0.0	0.0	5.0	
1	Afghanistan	Agrumes, Autres	vegetale	0.0	0.0	1.0	
2	Afghanistan	Aliments pour enfants	vegetale	0.0	0.0	1.0	
3	Afghanistan	Ananas	vegetale	0.0	0.0	0.0	
4	Afghanistan	Bananes	vegetale	0.0	0.0	4.0	

2.3 - Analyse exploratoire du fichier aide alimentaire

```
In [15]: #Afficher les dimensions du dataset
print("Le tableau comporte {} observation(s) ou article(s)".format(aide_alimentaire.shape[0]))
print("Le tableau comporte {} colonne(s)".format(aide_alimentaire.shape[1]))
```

Le tableau comporte 1475 observation(s) ou article(s)

Le tableau comporte 4 colonne(s)

```
In [16]: #Consulter le nombre de colonnes
print("nombre de colonnes:", aide_alimentaire.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1475 entries, 0 to 1474
Data columns (total 4 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Pays bénéficiaire     1475 non-null   object
1   Année                 1475 non-null   int64
2   Produit               1475 non-null   object
3   Valeur                1475 non-null   int64
dtypes: int64(2), object(2)
memory usage: 46.2+ KB
nombre de colonnes: None
```

```
In [17]: #Affichage des 5 premières lignes de la table
aide_alimentaire.head()
```

```
Out[17]:
```

	Pays bénéficiaire	Année	Produit	Valeur
0	Afghanistan	2013	Autres non-céréales	682
1	Afghanistan	2014	Autres non-céréales	335
2	Afghanistan	2013	Blé et Farin	39224
3	Afghanistan	2014	Blé et Farin	15160
4	Afghanistan	2013	Céréales	40504

```
In [18]: #changement du nom de la colonne Pays bénéficiaire par Zone
aide_alimentaire.rename(columns={'Pays bénéficiaire': 'Zone'}, inplace=True)
aide_alimentaire.head()
```

Out[18]:

	Zone	Année	Produit	Valeur
0	Afghanistan	2013	Autres non-céréales	682
1	Afghanistan	2014	Autres non-céréales	335
2	Afghanistan	2013	Blé et Farin	39224
3	Afghanistan	2014	Blé et Farin	15160
4	Afghanistan	2013	Céréales	40504

In [19]: *#Multiplication de la colonne Aide_alimentaire qui contient des tonnes par 1000 pour aide_alimentaire['Valeur'] = aide_alimentaire['Valeur']*1000*

In [20]: *#Affichage des 5 premières lignes de la table*
aide_alimentaire.head()

Out[20]:

	Zone	Année	Produit	Valeur
0	Afghanistan	2013	Autres non-céréales	682000
1	Afghanistan	2014	Autres non-céréales	335000
2	Afghanistan	2013	Blé et Farin	39224000
3	Afghanistan	2014	Blé et Farin	15160000
4	Afghanistan	2013	Céréales	40504000

2.3 - Analyse exploratoire du fichier sous nutrition

In [21]: *#Afficher les dimensions du dataset*
sous_nutrition.shape[0]

Out[21]: 1218

In [22]: *#Consulter le nombre de colonnes*
sous_nutrition.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1218 entries, 0 to 1217
Data columns (total 3 columns):
#   Column  Non-Null Count  Dtype
---  -
0   Zone    1218 non-null     object
1   Année   1218 non-null     object
2   Valeur  624 non-null      object
dtypes: object(3)
memory usage: 28.7+ KB
```

In [23]: *#Afficher les 5 premières lignes de la table*

```
sous_nutrition.head()
```

Out[23]:

	Zone	Année	Valeur
0	Afghanistan	2012-2014	8.6
1	Afghanistan	2013-2015	8.8
2	Afghanistan	2014-2016	8.9
3	Afghanistan	2015-2017	9.7
4	Afghanistan	2016-2018	10.5

In [24]: *#Conversion de La colonne sous nutrition en numérique*

```
sous_nutrition['Valeur'] = pd.to_numeric(sous_nutrition['Valeur'], errors= 'coerce')
```

In [25]: *#Conversion de La colonne (avec l'argument errors=coerce qui permet de convertir au #Puis remplacement des NaN en 0*

```
sous_nutrition['Valeur'] = sous_nutrition['Valeur'].fillna(0)
```

In [26]: *#changement du nom de La colonne Valeur par sous_nutrition*

```
sous_nutrition.rename(columns={'Valeur':'sous_nutrition'}, inplace= True)
```

In [27]: *#Multiplication de La colonne sous_nutrition par 1000000*

```
sous_nutrition['sous_nutrition'] = sous_nutrition['sous_nutrition'] * 1000000
```

In [28]: *#Afficher Les 5 premières lignes de La table*

```
sous_nutrition.head()
```

Out[28]:

	Zone	Année	sous_nutrition
0	Afghanistan	2012-2014	8600000.0
1	Afghanistan	2013-2015	8800000.0
2	Afghanistan	2014-2016	8900000.0
3	Afghanistan	2015-2017	9700000.0
4	Afghanistan	2016-2018	10500000.0

3.1 - Proportion de personnes en sous nutrition

In [29]: *# Il faut tout d'abord faire une jointure entre La table population et La table sou*
#Filtrer sur 2017 pour La population

```
pop_2017 = population[population['Année'] == 2017]
```

#Filtrer sur 2016-2018 pour La sous_nutrition

```
sous_nut_2017 = sous_nutrition[sous_nutrition['Année'] == '2016-2018']
```

#Faire la jointure entre Les deux tables

```
jointure= pd.merge(pop_2017, sous_nut_2017, on= 'Zone', how= 'left')
```

```
In [30]: #Affichage du dataset
jointure.head()
```

```
Out[30]:
```

	Zone	Année_x	Population	Année_y	sous_nutrition
0	Afghanistan	2017	36296113.0	2016-2018	10500000.0
1	Afrique du Sud	2017	57009756.0	2016-2018	3100000.0
2	Albanie	2017	2884169.0	2016-2018	100000.0
3	Algérie	2017	41389189.0	2016-2018	1300000.0
4	Allemagne	2017	82658409.0	2016-2018	0.0

```
In [31]: #Calcul et affichage du nombre de personnes en état de sous nutrition
nb_sous_nutritiion= jointure['sous_nutrition'].sum()
print("Le nombre de personnes en état de sous nutrition en 2017 est de :", nb_sous_
```

Le nombre de personnes en état de sous nutrition en 2017 est de : 535700000.0

3.2 - Nombre théorique de personne qui pourrait être nourries

```
In [32]: #Combien mange en moyenne un être humain ? Source => https://www.who.int/fr/news-r
moyenne_nourriture_par_personne_et_par_jour_kcal = 2500
moyenne_nourriture_par_personne_et_par_an_kcal = moyenne_nourriture_par_personne_et
print("En moyenne, un être humain mange {} kcal de nourriture par an.".format(moyen
```

En moyenne, un être humain mange 912500 kcal de nourriture par an.

```
In [33]: #On commence par faire une jointure entre Le data frame population et Dispo_aliment
population_pop= population[population['Année']==2017][['Zone', 'Population']]
jointure_pop_dispo= pd.merge(dispo_alimentaire, population_pop, on= 'Zone', how= 'l
jointure_pop_dispo.fillna(0, inplace= True)
```

```
In [34]: #Affichage du nouveau dataframe
jointure_pop_dispo.head()
```

Out[34]:

	Zone	Produit	Origine	Aliments pour animaux	Autres Utilisations	Disponibilité alimentaire (Kcal/personne/jour)	Dispo alimen c (kg/perso
0	Afghanistan	Abats Comestible	animale	0.0	0.0	5.0	
1	Afghanistan	Agrumes, Autres	vegetale	0.0	0.0	1.0	
2	Afghanistan	Aliments pour enfants	vegetale	0.0	0.0	1.0	
3	Afghanistan	Ananas	vegetale	0.0	0.0	0.0	
4	Afghanistan	Bananes	vegetale	0.0	0.0	4.0	

In [35]: *#Création de la colonne dispo_kcal avec calcul des kcal disponibles mondialement*
 jointure_pop_dispo['dispo_kcal'] = (jointure_pop_dispo['Disponibilité alimentaire (Kcal/personne/jour)'] * jointure_pop_dispo['Population (millions)']).sum()
 print(" le total des kcal disponibles mondialement est de ", jointure_pop_dispo['dispo_kcal'].sum())

le total des kcal disponibles mondialement est de 7635429388975815.0

In [36]: *#Calcul du nombre d'humains pouvant être nourris*
 nbre_humains_nourris = jointure_pop_dispo['dispo_kcal'].sum() / (moyenne_nourriture_personne_kcal)
 print("Le nombre d'humains pouvant être nourris avec les kcal disponibles est de :", nbre_humains_nourris)

Le nombre d'humains pouvant être nourris avec les kcal disponibles est de : 8367593850.9324

3.3 - Nombre théorique de personne qui pourrait être nourrie avec les produits végétaux

In [37]: *#Transfert des données avec Les végétaux dans un nouveau dataframe*
 jointure_pop_dispo_vegetaux = jointure_pop_dispo[jointure_pop_dispo['Origine'] == 'vegetale']
 jointure_pop_dispo_vegetaux.head()

Out[37]:

	Zone	Produit	Origine	Aliments pour animaux	Autres Utilisations	Disponibilité alimentaire (Kcal/personne/jour)	Dispon alimenta qu (kg/personn
1	Afghanistan	Agrumes, Autres	vegetale	0.0	0.0	1.0	
2	Afghanistan	Aliments pour enfants	vegetale	0.0	0.0	1.0	
3	Afghanistan	Ananas	vegetale	0.0	0.0	0.0	
4	Afghanistan	Bananes	vegetale	0.0	0.0	4.0	
6	Afghanistan	Bière	vegetale	0.0	0.0	0.0	

In [38]: *#Calcul du nombre de kcal disponible pour Les végétaux*
 nb_kcal_total_vegetaux= jointure_pop_dispo_vegetaux['dispo_kcal'].sum()
 print("Le nombre de kcal disponible pour les végétaux est de :", nb_kcal_total_vege

Le nombre de kcal disponible pour les végétaux est de : 6300178937197865.0

In [39]: *#Calcul du nombre d'humains pouvant être nourris avec Les végétaux*
 nbre_humains_nourris_vegetaux= nb_kcal_total_vegetaux/ (moyenne_nourriture_par_pers
 print("Le nombre d'humains pouvant être nourris avec les végétaux est de :", nbre_h

Le nombre d'humains pouvant être nourris avec les végétaux est de : 6904305684.6004

3.4 - Utilisation de la disponibilité intérieure

In [40]: *#Calcul de la disponibilité totale*
 dispo_totale= jointure_pop_dispo['Disponibilité intérieure'].sum()
 print("La disponibilité totale est de :", dispo_totale)

La disponibilité totale est de : 9848994000.0

In [41]: *#création d'une boucle for pour afficher Les différentes valeurs en fonction des co*
 colonnes_aliments= ['Aliments pour animaux', 'Pertes', 'Nourriture']
 for colonne in colonnes_aliments:
 total = dispo_alimentaire[colonne].sum()
 if colonne == 'Disponibilité intérieure':
 print(f"Disponibilité intérieure totale : {total:,.0f} tonnes")
 elif colonne == 'Nourriture':
 print(f"Alimentation humaine (Nourriture) : {total:,.0f} tonnes ({tota
 elif colonne == 'Aliments pour animaux':
 print(f"Aliments pour animaux : {total:,.0f} tonnes ({tota
 elif colonne == 'Pertes':
 print(f"Pertes : {total:,.0f} tonnes ({tota

Aliments pour animaux	: 1,304,245,000 tonnes (13.2 %)
Pertes	: 453,698,000 tonnes (4.6 %)
Alimentation humaine (Nourriture)	: 4,876,258,000 tonnes (49.5 %)

3.5 - Utilisation des céréales

In [42]: *#Création d'une liste avec toutes les variables*

```
liste_cereales = [
    'Blé',
    'Riz (Eq Blanchi)',
    'Orge',
    'Maïs',
    'Seigle',
    'Avoine',
    'Millet',
    'Sorgho',
    'Céréales, Autres'
]
```

In [43]: *#Création d'un dataframe avec les informations uniquement pour ces céréales*

```
df_cereales = dispo_alimentaire[dispo_alimentaire['Produit'].isin(liste_cereales)]
```

In [44]: *#Affichage de la proportion d'alimentation animale*

```
proportion_alimentation_animale = (df_cereales['Aliments pour animaux'].sum() / df_
print("La proportion d'alimentation animale pour les céréales est de : {:.2f}%".for
```

La proportion d'alimentation animale pour les céréales est de : 36.29%

In [45]: *#Affichage de la proportion d'alimentation animale*

```
total_cereales= df_cereales['Disponibilité intérieure'].sum()
alimentation_animale_cereales= df_cereales['Aliments pour animaux'].sum()
proportion_alimentation_animale = (alimentation_animale_cereales / total_cereales)
print("La proportion d'alimentation animale pour les céréales est de : {:.2f}%".for
```

La proportion d'alimentation animale pour les céréales est de : 36.29%

3.6 - Pays avec la proportion de personnes sous-alimentée la plus forte en 2017

In [46]: *#Création de la colonne proportion en pourcentage par pays*

```
jointure['Proportion sous alimentation'] = (jointure['sous_nutrition'] / jointure['
```

In [47]: *#affichage après tri des 10 pires pays*

```
jointure.sort_values(by='Proportion sous alimentation', ascending=False, inplace=True)
jointure.head(10)
```

Out[47]:

	Zone	Année_x	Population	Année_y	sous_nutrition	Proportion sous alimentation
87	Haïti	2017	10982366.0	2016-2018	5300000.0	48.259182
181	République populaire démocratique de Corée	2017	25429825.0	2016-2018	12000000.0	47.188685
128	Madagascar	2017	25570512.0	2016-2018	10500000.0	41.062924
122	Libéria	2017	4702226.0	2016-2018	1800000.0	38.279742
119	Lesotho	2017	2091534.0	2016-2018	800000.0	38.249438
216	Tchad	2017	15016753.0	2016-2018	5700000.0	37.957606
186	Rwanda	2017	11980961.0	2016-2018	4200000.0	35.055619
145	Mozambique	2017	28649018.0	2016-2018	9400000.0	32.810898
219	Timor-Leste	2017	1243258.0	2016-2018	400000.0	32.173531
0	Afghanistan	2017	36296113.0	2016-2018	10500000.0	28.928718

3.7 - Pays qui ont le plus bénéficié d'aide alimentaire depuis 2013

```
In [48]: #calcul du total de l'aide alimentaire par pays
aide_alimentaire_par_pays= aide_alimentaire.groupby('Zone')['Valeur'].sum().reset_i
```

```
In [49]: #affichage après trie des 10 pays qui ont bénéficié Le plus de l'aide alimentaire
aide_alimentaire_par_pays.sort_values(by='Valeur', ascending=False, inplace=True)
aide_alimentaire_par_pays.head(10)
```

Out[49]:

	Zone	Valeur
50	République arabe syrienne	1858943000
75	Éthiopie	1381294000
70	Yémen	1206484000
61	Soudan du Sud	695248000
60	Soudan	669784000
30	Kenya	552836000
3	Bangladesh	348188000
59	Somalie	292678000
53	République démocratique du Congo	288502000
43	Niger	276344000

3.8 - Evolution des 5 pays qui ont le plus bénéficiés de l'aide alimentaire entre 2013 et 2016

```
In [50]: #Création d'un dataframe avec la zone, l'année et l'aide alimentaire puis groupby s
aide_alimentaire_grouped= aide_alimentaire.groupby(['Zone', 'Année'])['Valeur'].sum
#Affichage du dataframe
aide_alimentaire_grouped.head(20)
```

Out[50]:

	Zone	Année	Valeur
0	Afghanistan	2013	128238000
1	Afghanistan	2014	57214000
2	Algérie	2013	35234000
3	Algérie	2014	18980000
4	Algérie	2015	17424000
5	Algérie	2016	9476000
6	Angola	2013	5000000
7	Angola	2014	14000
8	Bangladesh	2013	131018000
9	Bangladesh	2014	194628000
10	Bangladesh	2015	22542000
11	Bhoutan	2013	1724000
12	Bhoutan	2014	146000
13	Bhoutan	2015	578000
14	Bhoutan	2016	218000
15	Bolivie (État plurinational de)	2014	6000
16	Burkina Faso	2013	18620000
17	Burkina Faso	2014	22938000
18	Burkina Faso	2015	23182000
19	Burkina Faso	2016	72000

In [51]: *#Création d'une liste contenant les 5 pays qui ont le plus bénéficiées de l'aide al*

```
top_5_pays_aide= aide_alimentaire_par_pays.head(5)['Zone'].tolist()
print("Les 5 pays qui ont le plus bénéficiées de l'aide alimentaire sont :", top_5_
```

Les 5 pays qui ont le plus bénéficiées de l'aide alimentaire sont : ['République arabe syrienne', 'Éthiopie', 'Yémen', 'Soudan du Sud', 'Soudan']

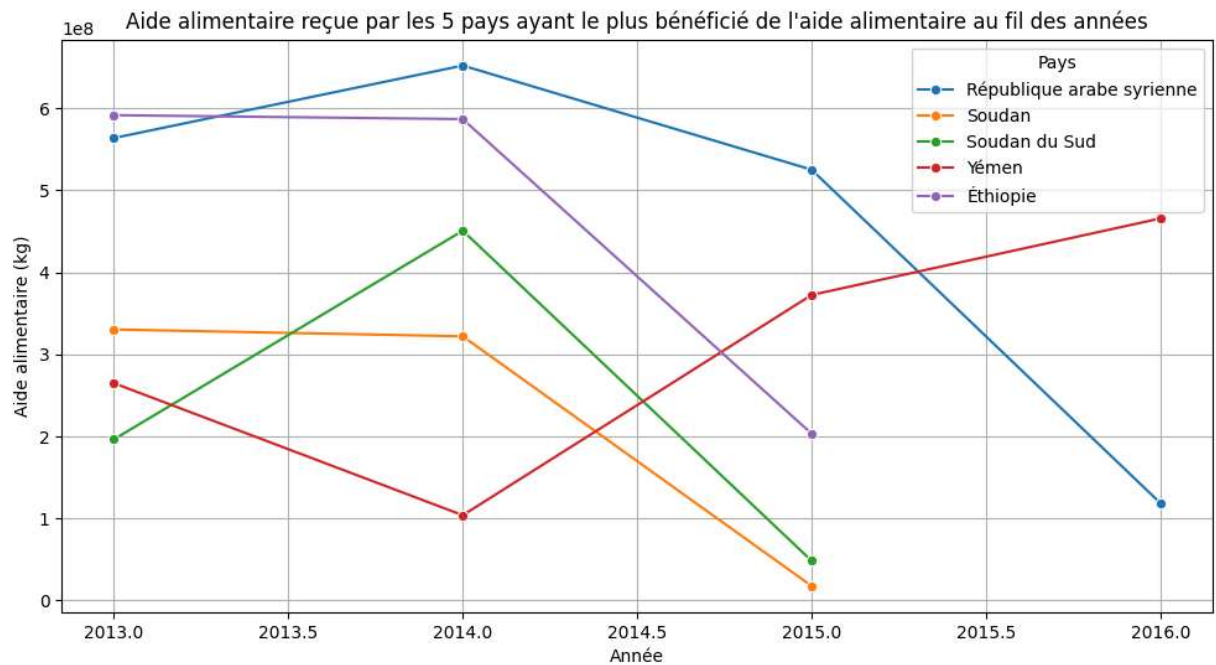
In [52]: *#On filtre sur le dataframe avec notre liste*

```
aide_alimentaire_top_5 = aide_alimentaire_grouped[aide_alimentaire_grouped['Zone'].
aide_alimentaire_top_5.head(15)
```

Out[52]:

	Zone	Année	Valeur
0	République arabe syrienne	2013	563566000
1	République arabe syrienne	2014	651870000
2	République arabe syrienne	2015	524949000
3	République arabe syrienne	2016	118558000
4	Soudan	2013	330230000
5	Soudan	2014	321904000
6	Soudan	2015	17650000
7	Soudan du Sud	2013	196330000
8	Soudan du Sud	2014	450610000
9	Soudan du Sud	2015	48308000
10	Yémen	2013	264764000
11	Yémen	2014	103840000
12	Yémen	2015	372306000
13	Yémen	2016	465574000
14	Éthiopie	2013	591404000

```
In [53]: # Affichage des pays avec l'aide alimentaire par année
plt.figure(figsize=(12, 6))
sns.lineplot(data=aide_alimentaire_top_5, x='Année', y='Valeur', hue='Zone', marker='o')
plt.title("Aide alimentaire reçue par les 5 pays ayant le plus bénéficié de l'aide")
plt.xlabel("Année")
plt.ylabel("Aide alimentaire (kg)")
plt.legend(title='Pays')
plt.grid()
plt.show()
```



3.9 - Pays avec le moins de disponibilité par habitant

```
In [54]: #Calcul de la disponibilité en kcal par personne par jour par pays
dispo_par_pays = dispo_alimentaire.groupby('Zone')['Disponibilité alimentaire (Kcal
dispo_par_pays = dispo_par_pays.rename(columns={'Disponibilité alimentaire (Kcal/pe
```

```
In [55]: #Affichage des 10 pays qui ont le moins de dispo alimentaire par personne
bottom_10 = dispo_par_pays.sort_values('Dispo_kcal_pers_jour').head(10)
print("10 pays avec le moins de disponibilité alimentaire par habitant (kcal/pers/j
display(bottom_10)
```

10 pays avec le moins de disponibilité alimentaire par habitant (kcal/pers/jour) :

	Zone	Dispo_kcal_pers_jour
128	République centrafricaine	1879.0
166	Zambie	1924.0
91	Madagascar	2056.0
0	Afghanistan	2087.0
65	Haïti	2089.0
133	République populaire démocratique de Corée	2093.0
151	Tchad	2109.0
167	Zimbabwe	2113.0
114	Ouganda	2126.0
172	Éthiopie	2129.0

3.10 - Pays avec le plus de disponibilité par habitant

```
In [56]: #Affichage des 10 pays qui ont le plus de dispo alimentaire par personne
top_10 = dispo_par_pays.sort_values('Dispo_kcal_pers_jour', ascending=False).head(10)
print("10 pays avec le plus de disponibilité alimentaire par habitant (kcal/pers/jour)")
display(top_10)
```

10 pays avec le plus de disponibilité alimentaire par habitant (kcal/pers/jour) :

	Zone	Dispo_kcal_pers_jour
11	Autriche	3770.0
16	Belgique	3737.0
159	Turquie	3708.0
171	États-Unis d'Amérique	3682.0
74	Israël	3610.0
72	Irlande	3602.0
75	Italie	3578.0
89	Luxembourg	3540.0
168	Égypte	3518.0
4	Allemagne	3503.0

3.11 - Exemple de la Thaïlande pour le Manioc

```
In [57]: #création d'un dataframe avec uniquement La Thaïlande
thaïlande = dispo_alimentaire[dispo_alimentaire['Zone'] == 'Thaïlande']
```

```
In [58]: #Calcul de la sous nutrition en Thaïlande
# (Utilise l'intervalle 2016-2018 pour approx 2017, comme pour les autres questions)
sous_nut_thai = sous_nutrition[(sous_nutrition['Année'] == '2016-2018') & (sous_nut
pop_thai = population[(population['Année'] == 2017) & (population['Zone'] == 'Thaïl
nb_sous_nut_thai = sous_nut_thai['sous_nutrition'].values[0]
print("Le nombre de personnes en état de sous nutrition en Thaïlande en 2017 est de
prop_sous_nut_thai = (nb_sous_nut_thai / pop_thai) * 100
print("En Thaïlande, la proportion de personnes en état de sous nutrition en 2017 e
```

Le nombre de personnes en état de sous nutrition en Thaïlande en 2017 est de : 62000
00.0

En Thaïlande, la proportion de personnes en état de sous nutrition en 2017 est de :
8.96%

```
In [59]: # On calcule la proportion exportée en fonction de la proportion
manioc_thai = thaïlande[thaïlande['Produit'] == 'Manioc']
prop_export_manioc = (manioc_thai['Exportations - Quantité'].values[0] / manioc_tha
print("En Thaïlande, la proportion de manioc exportée est de : {:.2f}%".format(prop
```

En Thaïlande, la proportion de manioc exportée est de : 83.41%

Etape 6 - Analyse complémentaires

```
In [60]: #ajouter en dessous toutes les analyses complémentaires

# 1. Disponibilité en protéines : top 10 max/min (similaire à kcal)
dispo_prot_par_pays = dispo_alimentaire.groupby('Zone')['Disponibilité de protéines
dispo_prot_par_pays = dispo_prot_par_pays.rename(columns={'Disponibilité de protéin

top_10_prot = dispo_prot_par_pays.sort_values('Dispo_prot_g_pers_jour', ascending=F
bottom_10_prot = dispo_prot_par_pays.sort_values('Dispo_prot_g_pers_jour').head(10)

print("Analyse complémentaire 1 : Top 10 pays avec le plus de disponibilité en prot
display(top_10_prot)

print("Bottom 10 pays avec le moins de disponibilité en protéines (g/pers/jour) :")
display(bottom_10_prot)

# 2. Corrélation aide alimentaire totale (2013-2016) et sous-nutrition (2017)
aide_totale = aide_alimentaire.groupby('Zone')['Valeur'].sum().reset_index()
sous_nut_2017 = sous_nutrition[sous_nutrition['Année'] == '2016-2018'][['Zone', 'so
corr_aide_sousnut = aide_totale.merge(sous_nut_2017, on='Zone', how='inner').sort_v

print("\nAnalyse complémentaire 2 : Top 10 pays aidés vs. sous-nutrition (aide en t
```

```

display(corr_aide_sousnut)

# 3. Pertes totales mondiales
pertes_totales = dispo_alimentaire['Pertes'].sum()

print("\nAnalyse complémentaire 3 : Pertes alimentaires totales mondiales :")
print(f"{pertes_totales:,.0f} tonnes (soit environ 4.6% de la disponibilité intérieure)")

# 4. Optionnel : Graphique simple (ex. barplot top 10 pertes par pays)
pertes_par_pays = dispo_alimentaire.groupby('Zone')['Pertes'].sum().sort_values(ascending=True)
plt.figure(figsize=(10,6))
sns.barplot(x=pertes_par_pays.index, y=pertes_par_pays.values, hue=pertes_par_pays.index)
plt.title('Top 10 pays avec le plus de pertes alimentaires (tonnes)')
plt.xlabel('Pertes (tonnes)')
plt.ylabel('Pays')
plt.grid()
plt.show()

```

Analyse complémentaire 1 : Top 10 pays avec le plus de disponibilité en protéines (g/pers/jour) :

	Zone	Dispo_prot_g_pers_jour
73	Islande	133.06
33	Chine - RAS de Hong-Kong	129.07
74	Israël	128.00
88	Lituanie	124.36
94	Maldives	122.32
52	Finlande	117.56
89	Luxembourg	113.64
102	Monténégro	111.90
119	Pays-Bas	111.46
2	Albanie	111.37

Bottom 10 pays avec le moins de disponibilité en protéines (g/pers/jour) :

	Zone	Dispo_prot_g_pers_jour
87	Libéria	37.66
62	Guinée-Bissau	44.05
103	Mozambique	45.68
128	République centrafricaine	46.04
91	Madagascar	46.69
65	Haïti	47.70
167	Zimbabwe	48.32
39	Congo	51.41
114	Ouganda	52.64
139	Sao Tomé-et-Principe	53.10

Analyse complémentaire 2 : Top 10 pays aidés vs. sous-nutrition (aide en tonnes) :

	Zone	Valeur	sous_nutrition
50	République arabe syrienne	1858943000	0.0
74	Éthiopie	1381294000	21100000.0
69	Yémen	1206484000	0.0
61	Soudan du Sud	695248000	0.0
60	Soudan	669784000	5000000.0
30	Kenya	552836000	11900000.0
3	Bangladesh	348188000	21500000.0
59	Somalie	292678000	0.0
53	République démocratique du Congo	288502000	0.0
43	Niger	276344000	0.0

Analyse complémentaire 3 : Pertes alimentaires totales mondiales :
453,698,000 tonnes (soit environ 4.6% de la disponibilité intérieure)

